

Sistemas en Tiempo Real

3º Curso – Grado en Ingeniería Informática
Especialidad Ingeniería de Computadores



Sistemas en Tiempo Real

Programa de Teoría

Tema 1: Introducción a los Sistemas en Tiempo Real (2 h.)

Tema 2: Lenguajes para aplicaciones en Tiempo Real (2 h.)

Tema 3: Interfaces y Elementos Hardware (4 h.)

Tema 4: Concurrencia y Sincronización (6 h.)

Tema 5: Sistemas Operativos en Tiempo Real (6 h.)

Tema 6: Planificación de Tiempo Real (6 h.)



Tema 5. Sistemas Operativos de Tiempo Real



1. Estructura de los Sistemas de Control

1. Un sistema operativo sencillo
2. Diferencias entre un S.O. Multiusuario y un S.O. Multitarea
3. Estructura general de un S.O. en Tiempo Real

2. Administrador de Tareas

1. Administrador de Memoria

1. Procesos e Hilos

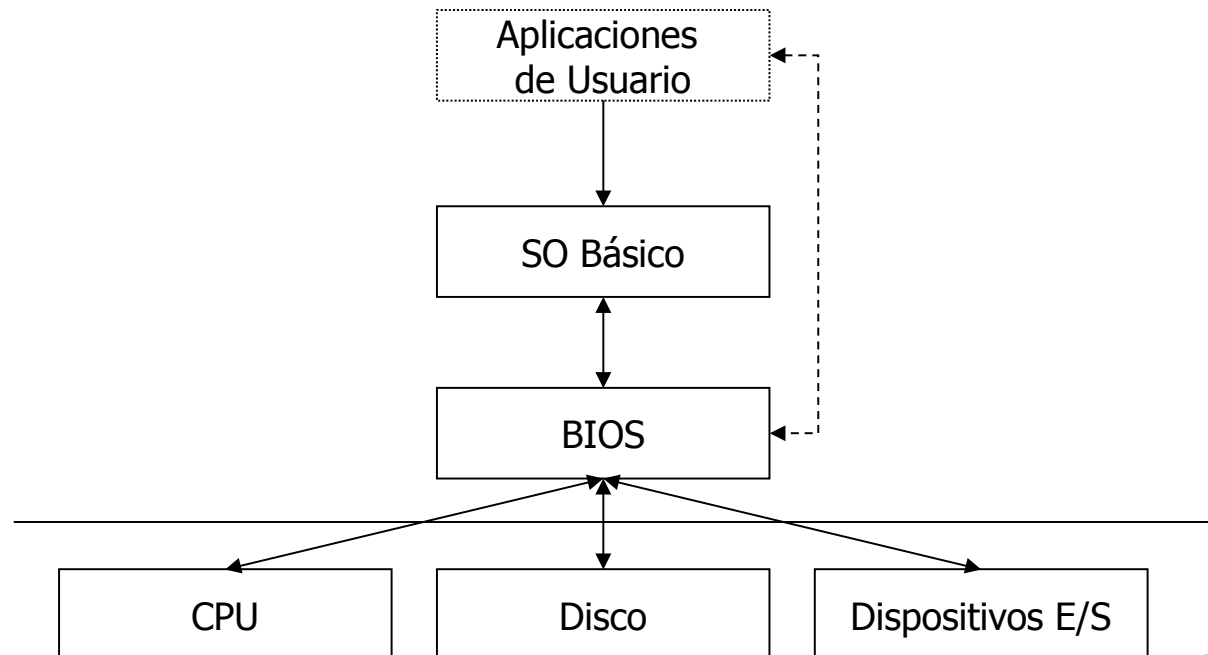
2. Comunicación y sincronización entre procesos

3. Control de recursos



Tema 5. Sistemas Operativos de Tiempo Real

- Un sistema operativo sencillo
 - Los S.O. de control suelen ser sencillos, permitir la interconexión de los programas de usuario con el hardware.
 - Algunos S.O. permiten el acceso directo a rutinas de bajo nivel que acceden al hardware.



Tema 5. Sistemas Operativos de Tiempo Real

• Componentes de un S.O. de Control sencillo

- Las aplicaciones de usuario se comunican con el S.O. Básico (BOS) para interactuar con el hardware.
- El BOS se comunica con el Hardware a través de un conjunto de rutinas implementadas en bajo nivel: BIOS (Basic Input Output System).
- El BIOS contiene los drivers de los dispositivos que manipulan los dispositivos físicos.
- En algunos casos, las aplicaciones de usuario pueden "saltarse" el BOS para solicitar servicios directamente al BIOS.
- La conexión entre lenguajes de alto nivel y el S.O. lo realiza el compilador a través de rutinas *run-time*.



Tema 5. Sistemas Operativos de Tiempo Real

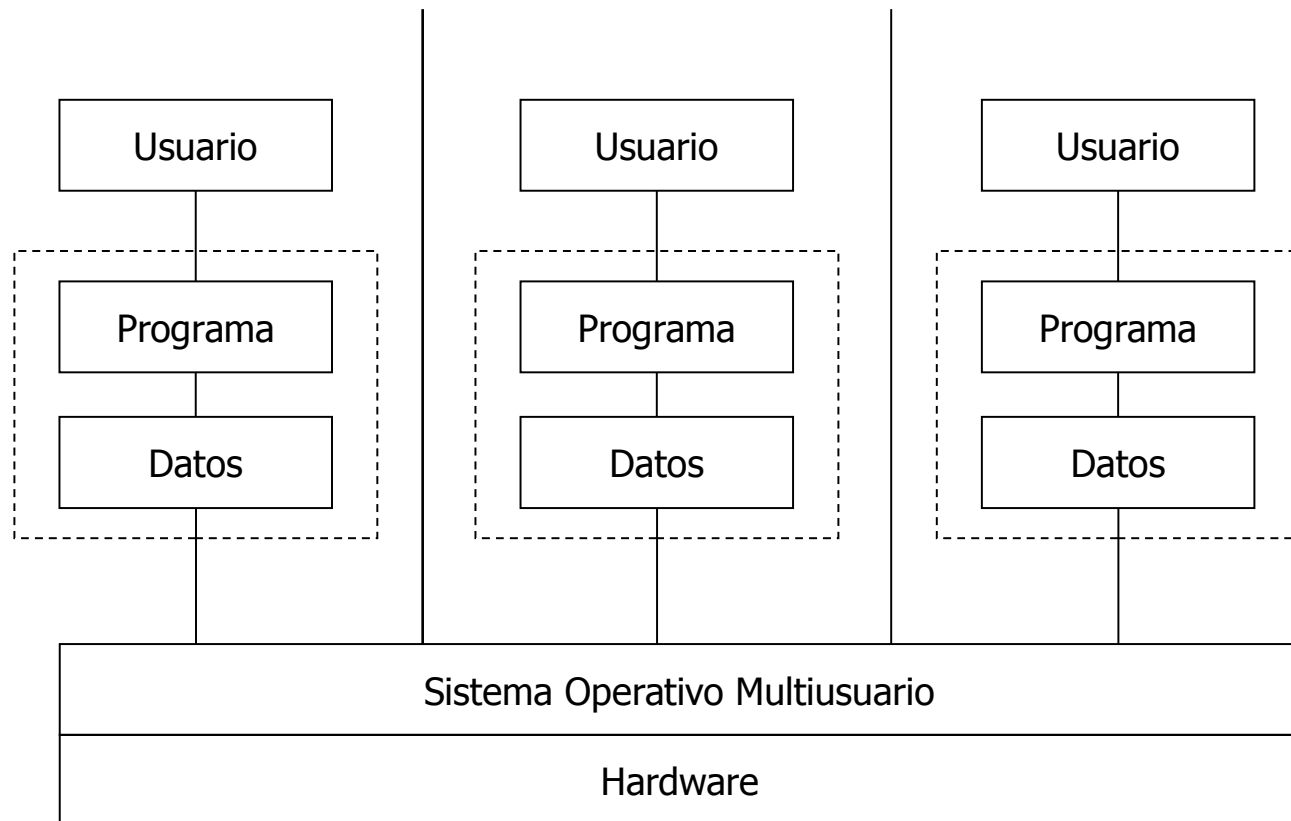
• Sistema Operativo Multiusuario

- Los sistemas multiusuario presenta diferentes usuarios que ejecutan un único programa independiente cada uno de ellos.
 - El S.O. permite a cada usuario ejecutar un programa como si todos los recursos del sistema estuviesen asignados para él.
 - La comunicación entre tareas (que serán de diferentes usuarios) se realiza a través del S.O.
 - No existen variables compartidas.
 - En un instante determinado no se puede saber qué usuario estará haciendo uso de la CPU o qué código estará en memoria, ni cuál entrará después.
 - El tiempo de la CPU se repartirá entre cada usuario.
 - El programa de cada usuario se ejecuta en un entorno protegido de tal forma que no interfiere uno con otro.



Tema 5. Sistemas Operativos de Tiempo Real

• Sistema Operativo Multiusuario



Tema 5. Sistemas Operativos de Tiempo Real

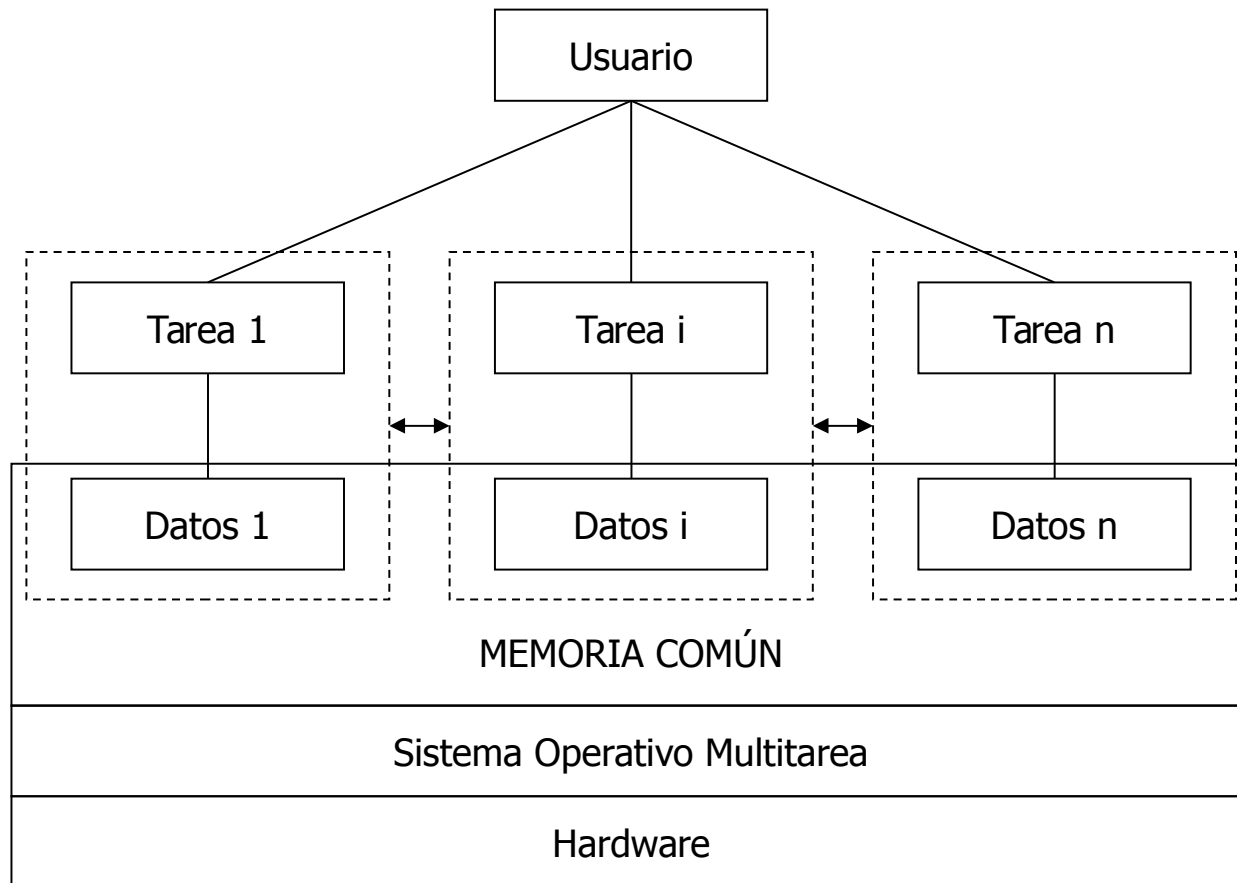
• Sistema Operativo Multitarea

- Hay un único usuario en el sistema que ejecuta diversas tareas que interactúan entre sí.
 - La interacción entre tareas se realiza mediante datos o variables compartidas.
 - Las tareas deberán acceder a las variables garantizando un acceso coherente. El S.O. solo proporcionará mecanismos para poder dar soporte a la exclusión mutua, pero no gestionará el acceso en sí.
 - Las tareas no tienen protección de memoria, es decir, que desde una tarea se podría acceder a los datos privados de otra tarea.
 - El tiempo de la CPU se repartirá entre cada una de las tareas.



Tema 5. Sistemas Operativos de Tiempo Real

- Sistema Operativo Multitarea



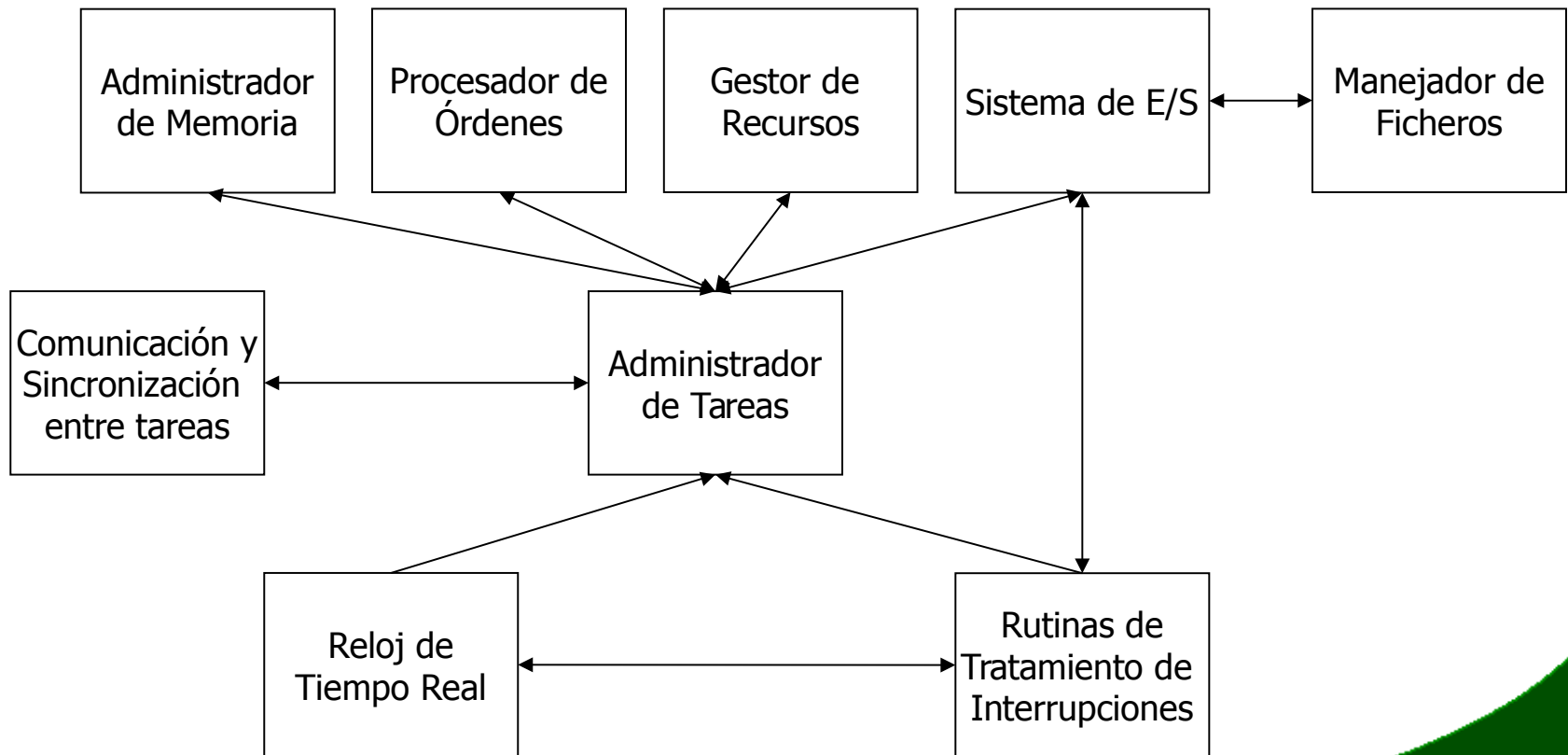
Tema 5. Sistemas Operativos de Tiempo Real

- Estructura general de un S.O. en Tiempo Real
 - Los S.O. deben asignar los recursos del sistema entre todas las tareas y usuarios, en un SOTR esta asignación es mucho más compleja debido a la presencia de tareas críticas y diferentes niveles de prioridad.
 - Los SOTR deben ser capaces de asignar prioridades a las tareas y planificar el tiempo de CPU utilizando algún esquema de prioridad.
 - Los SOTR deben permitir la utilización de interrupciones proporcionando un sistema útil y sencillo, a la vez que potente y robusto, para añadir/eliminar rutinas RTI propias.
 - Los SOTR deben proporcionar mecanismos que impidan que las tareas intenten utilizar un recurso al mismo tiempo.



Tema 5. Sistemas Operativos de Tiempo Real

• Estructura general de un S.O. en Tiempo Real



Tema 5. Sistemas Operativos de Tiempo Real

- Estructura general de un S.O. en Tiempo Real
 - Administrador de Tareas
 - Asigna la memoria y el tiempo de CPU a las tareas. Este módulo es el núcleo del sistema y también es llamado módulo monitor o *executive*.
 - Administrador de Memoria
 - Realiza el control de la memoria asignada dinámicamente a las tareas según lo solicitan.
 - Comunicación y Sincronización de Tareas
 - Permite la comunicación entre las tareas presentes en el sistema, además proporciona los mecanismos necesarios para la sincronización.
 - Gestor de Recursos
 - Realiza el control de todos los recursos del sistema excepto el tiempo de CPU y la memoria.



Tema 5. Sistemas Operativos de Tiempo Real

1. Estructura de los Sistemas de Control



2. Administrador de Tareas

1. Funciones del administrador de tareas
2. Estructura interna: Planificador y Despachador
3. Estados de las tareas
4. Descriptores de las tareas

1. Administrador de Memoria

1. Procesos e Hilos
2. Comunicación y sincronización entre procesos
3. Control de recursos



Tema 5. Sistemas Operativos de Tiempo Real

• Funciones del administrador de tareas

- El S.O. es la tarea de más alta prioridad en cualquier sistema. Toma control cada vez que hay que procesar una interrupción o gestionar una llamada al sistema por parte de las tareas de usuario.
- El núcleo fundamental se llama administrador de Tareas, módulo monitor o *executive*.
- El núcleo tiene las siguientes tareas:
 - Almacenar y mantener el estado de cada tarea.
 - Planificar el tiempo de CPU para cada tarea según la política de planificación y el tipo de tarea.
 - Realizar el cambio de contexto (guardar el estado de la tarea que estaba utilizando la CPU y cargar el estado de la tarea que va a utilizar la CPU, y proceder a la ejecución de la nueva tarea).



Tema 5. Sistemas Operativos de Tiempo Real

- **Funciones del administrador de tareas - II**
 - Desde el punto de vista del usuario el administrador de tareas es el encargado de:
 - Crear las tareas (definir las estructuras en memoria que dan soporte software a las tareas dentro del sistema e ingresarlas dentro de las diversas colas del sistema).
 - Destruir las tareas (eliminación del sistema y liberación de la memoria una vez han finalizado su ejecución).
 - Soporte de las estrategias de planificación para la asignación de tiempo de CPU a cada tarea.
 - La creación y destrucción de tareas son llamadas al sistema implementadas en la interfaz del S.O. y del lenguaje de programación.



Tema 5. Sistemas Operativos de Tiempo Real

- Estructura interna: Planificador y Despachador
 - El administrador de tareas está subdividido en 2 partes:
 - Planificador (*scheduler*)
 - Determina qué tarea se ejecutará y mantiene el estado de las tareas en el sistema.
 - Despachador o Cargador (*dispatcher/loader*)
 - Realiza la conmutación de tareas.
 - El administrador de tareas se ejecuta cada vez que se produce una interrupción de reloj de sistema. En esa interrupción se determina si hay que continuar ejecutando la tarea actual o si hay que cambiar de tarea.
 - En caso de determinar que hay que cambiar, se llama al planificador y una vez esté determinada la tarea a conmutar, se llama al despachador.



Tema 5. Sistemas Operativos de Tiempo Real

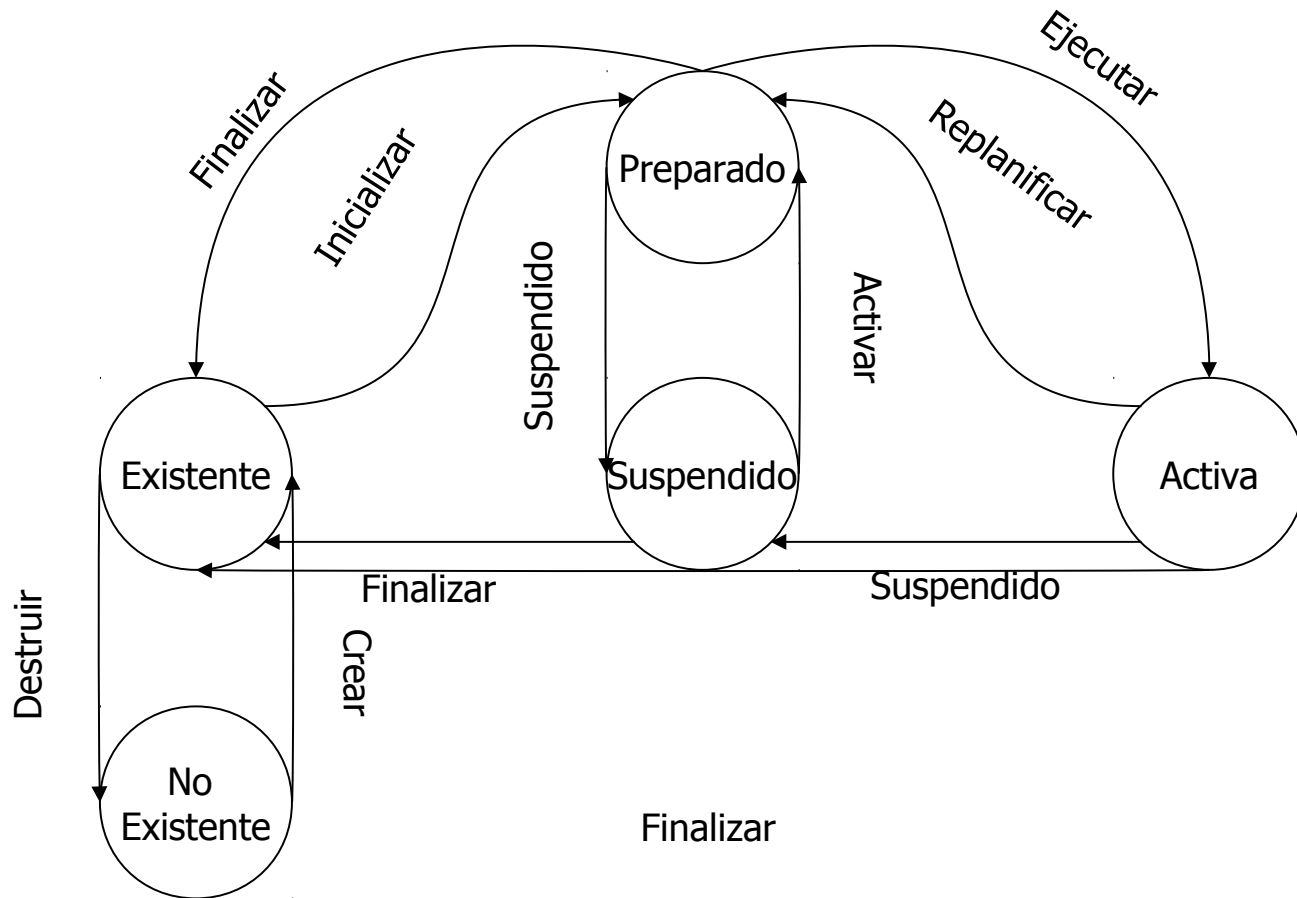
Estados de las tareas

- El planificador tiene que controlar el estado de ejecución de las tareas para determinar cuál será la siguiente que se deberá ejecutar o cuál debe ser la primera en desbloquearse.
- Las tareas se suelen dividir en:
 - Activas o En Ejecución.
 - Tarea que está ejecutándose. Si es un multiprocesador, habrá más de una.
 - Preparadas o Listas.
 - Tareas que pueden ejecutarse y están esperando su turno para entrar a ejecutarse.
 - Suspendida o En Espera.
 - Tareas que están esperando a que algún recurso esté disponible.
 - Existentes.
 - La tarea existe pero aún no tiene prioridad asignada.
 - No Existentes o Dormidas.
 - El sistema ha reservado memoria para la tarea aunque aún no se ha asignado código de ejecución. Tiene utilidad para Sistemas Operativos con limitación en el número máximo de tareas.



Tema 5. Sistemas Operativos de Tiempo Real

Estados de las tareas



Tema 5. Sistemas Operativos de Tiempo Real

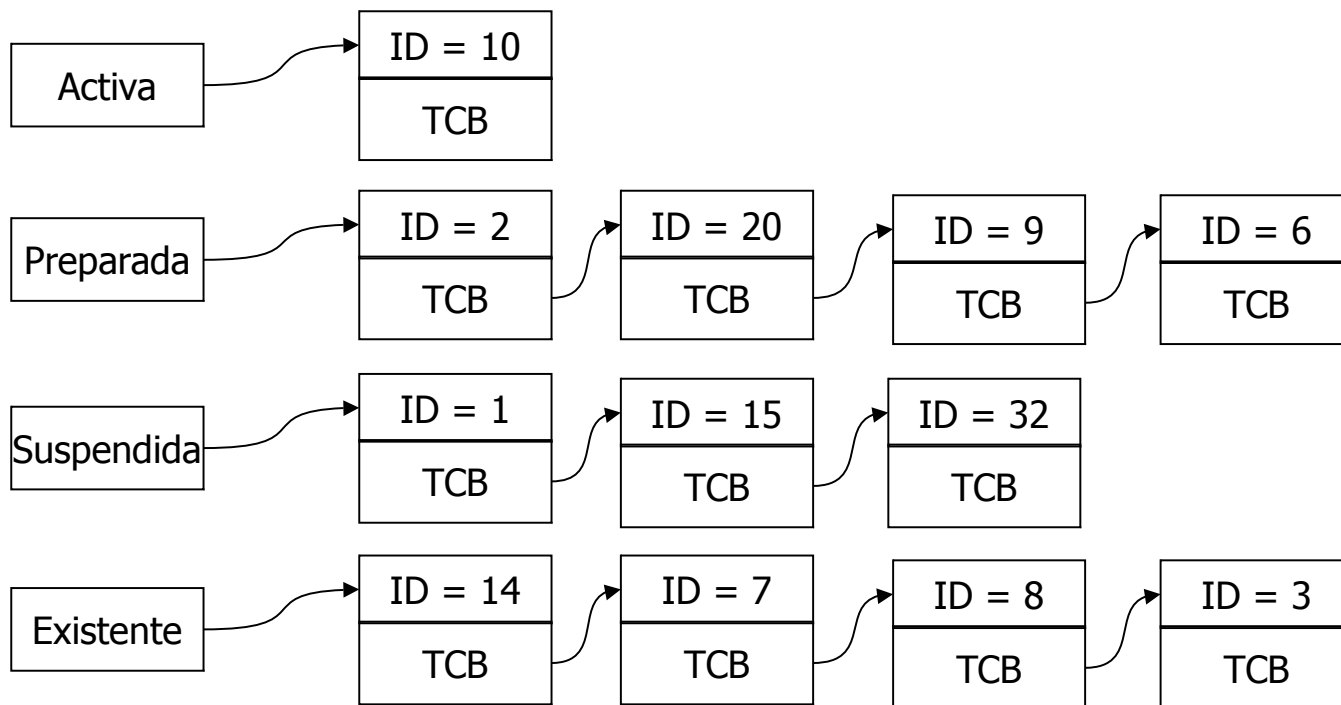
• Descriptores de las tareas

- El estado de cada tarea la almacena el S.O. en un bloque de memoria denominado *Descriptor de Tareas* (TD) o *Bloque de Control de Tareas* (TCB).
- Las tareas se almacenan en un conjunto de listas enlazadas. Hay una lista por cada estado posible.
- Al construirse con listas enlazadas en lugar de con una tabla de descriptores de tamaño fijo, no está restringido a un número máximo de tareas.
- El uso de listas enlazadas proporciona rapidez en los intercambios al no necesitar realizar copias, sino cambios en los punteros.
- En el TCB se almacenan:
 - El identificador único de la tarea.
 - La prioridad de la tarea.
 - El estado de la tarea.
 - Una zona de memoria en la que se guarda el contexto de la tarea o un puntero a una zona de memoria externa.
 - Un puntero al siguiente TCB de la lista.



Tema 5. Sistemas Operativos de Tiempo Real

● Descriptores de las tareas



Tema 5. Sistemas Operativos de Tiempo Real

1. Estructura de los Sistemas de Control

2. Administrador de Tareas

1. Administrador de Memoria

1. Funciones del administrador de Memoria

2. Administración básica de memoria

3. Reserva dinámica de memoria

4. Intercambio de memoria

5. Intercambio dentro de un programa: encadenamiento y solapamiento

1. Procesos e Hilos

2. Comunicación y sincronización entre procesos

3. Control de recursos



Tema 5. Sistemas Operativos de Tiempo Real

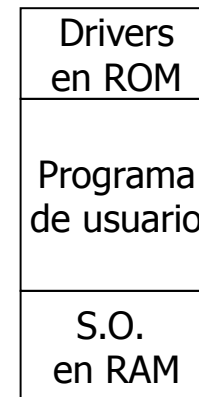
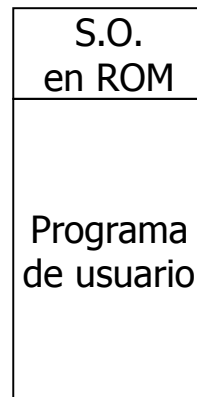
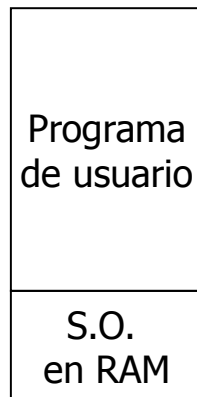
• Funciones del Administrador de Memoria

- La mayoría de SOTR plantean sistemas de software estático: todas las tareas necesarias se construyen en tiempo de compilación y no aparece o desaparece ninguna en tiempo de ejecución.
- En el caso que el SOTR admita la creación de nuevas tareas o su eliminación, o en el caso que el SOTR admita la reserva de memoria dinámica o el realojamiento de las tareas en otras zonas de memoria, es necesaria la presencia de un sistema de administración de la memoria.
- Este sistema:
 - Gestiona la memoria libre y ocupada del sistema.
 - Unifica todos los bloques de memoria libre para hacerlos adyacentes.
 - Se encarga de intercambiar tareas o trozos de tareas (datos o código) conforme se van necesitando entre memoria principal y auxiliar.
 - Gestiona la memoria dinámica.
 - Controla la memoria virtual y el "swapping".



Tema 5. Sistemas Operativos de Tiempo Real

- **Administración básica de Memoria**
 - Gestión sin intercambio ni paginación
 - Útil en sistemas con un único proceso que consume toda la memoria.

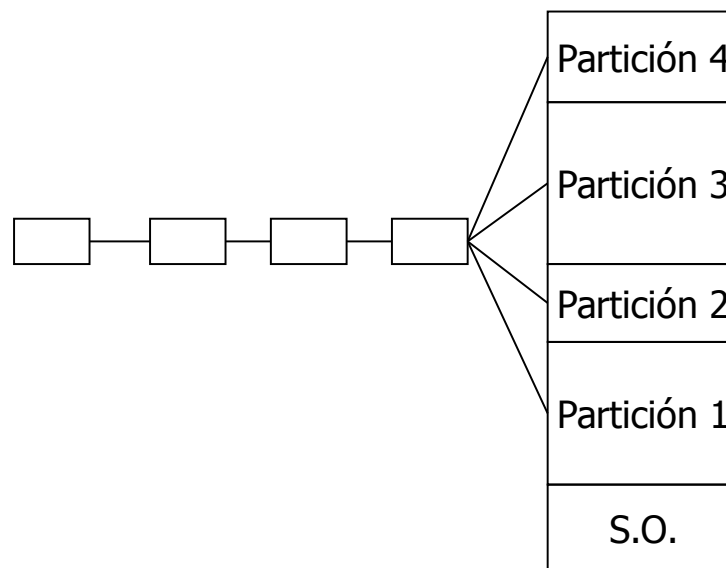
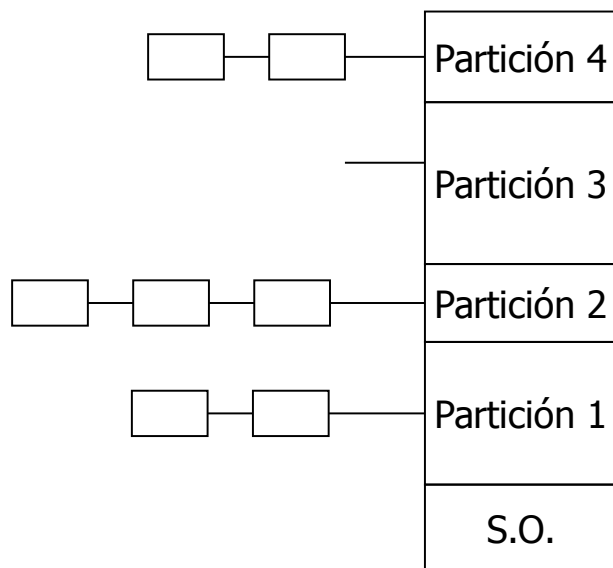


Tema 5. Sistemas Operativos de Tiempo Real

Administración básica de Memoria

Múltiples procesos en particiones fijas

- Útil para sistemas basados en procesamiento por lotes. La memoria se divide en varias particiones de tamaño fijo y diferente entre sí.
- Puede existir varias colas asociadas cada una a una partición con una lista de posibles tareas que van rotando según se necesita su carga en memoria.
- Otra opción es una única cola que pueda dirigir una tarea a cualquier partición.



Tema 5. Sistemas Operativos de Tiempo Real

• Administración básica de Memoria

• Reubicación y protección

- Asociado a la posibilidad de cargar una tarea en diferentes particiones aparece el concepto de reubicación.
- Las tareas se pueden ejecutar en distintas posiciones de memoria: las referencias de salto y carga han de ser relativas al inicio de la tarea.
- Para evitar saltos fuera del rango asociado a cada tarea, el sistema hardware debe proporcionar un mecanismo de protección (usualmente basado en códigos de protección que se comprueban vía hardware en cada acceso).
- La reubicación y la protección se pueden controlar vía hardware con registros de base y de límite.



Tema 5. Sistemas Operativos de Tiempo Real

• Intercambio de Memoria

- En sistemas de tiempo compartido suele haber más usuarios que memoria para todos sus procesos.
- Solución: mantener en disco los procesos que no caben en memoria y cargarlos conforme se van necesitando para su ejecución.
- La operación de mover procesos entre el disco y la memoria se denomina *intercambio* o *swapping*.
- El intercambio de memoria se puede realizar con particiones fijas, pero no es habitual y se usa la técnica de particiones variables.



Tema 5. Sistemas Operativos de Tiempo Real

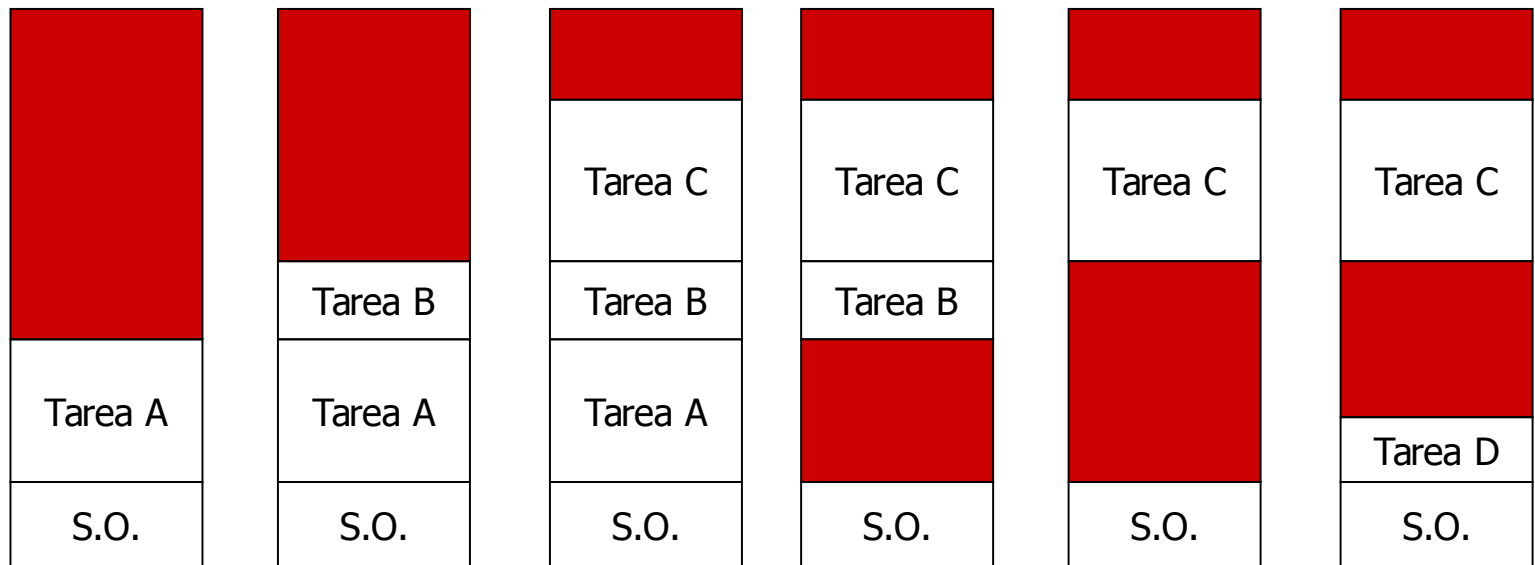
• Particiones variables

- Las particiones fijas malgastan mucha memoria y pueden impedir la ejecución de tareas aunque el espacio completo libre sea mucho mayor que el que requiere la tarea.
- Las particiones variables permiten variar dinámicamente el número y el tamaño de las tareas.
- Mejora la utilización, pero complica la asignación y desocupación de la memoria y el seguimiento del estado de cada zona de memoria.
- Puede provocar fragmentación de la memoria.



Tema 5. Sistemas Operativos de Tiempo Real

- Particiones variables



Tema 5. Sistemas Operativos de Tiempo Real

• Reserva dinámica de Memoria

- Las tareas pueden programarse con un número limitado variables de tamaño fijo, pero el uso de variables que puedan variar dinámicamente en tamaño o en número facilita la programación.
- Si las tareas no van a requerir más memoria que la original, se les puede asignar una partición con un tamaño determinado (que no va a variar a lo largo de la ejecución).
- Si las tareas van a hacer uso de memoria dinámica hay que concederles más memoria.
 - Si hay un hueco junto a la partición, se le asigna a la tarea y continúa.
 - Si no hay huecos "ceranos" a la partición, habrá que buscarlos o mover al resto de tareas para compactar la memoria y "abrir hueco".
- Se suele utilizar la técnica de reserva de bloques de datos contiguos con espacio para crecer (pila hacia abajo, datos/*heap* hacia arriba).



Tema 5. Sistemas Operativos de Tiempo Real

- Reserva dinámica de Memoria



Tema 5. Sistemas Operativos de Tiempo Real

• Gestión de la memoria dinámica

- En los SOTR se debe evitar la reserva dinámica siempre que se pueda, porque el beneficio es menor que el coste computacional.
=> Añadir más RAM frente a mayor complejidad en los algoritmos de gestión de la memoria dinámica.
- Hay tres métodos:
 - Mapa de bits
 - Listas enlazadas
 - Método de los colegas
- Mapa de bits
 - Se tiene una tabla donde cada bit representa un bloque de memoria de n bytes. Si es 1 está ocupada, si es 0 está libre.
 - La asignación se consigue buscando una ristra de k bits a 0 seguidos. La liberación se hace poniendo a 0 una serie de bits a 1.
 - La búsqueda de huecos es lenta.



Tema 5. Sistemas Operativos de Tiempo Real

• Gestión de la memoria dinámica

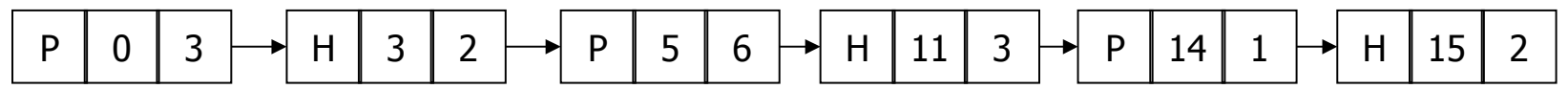
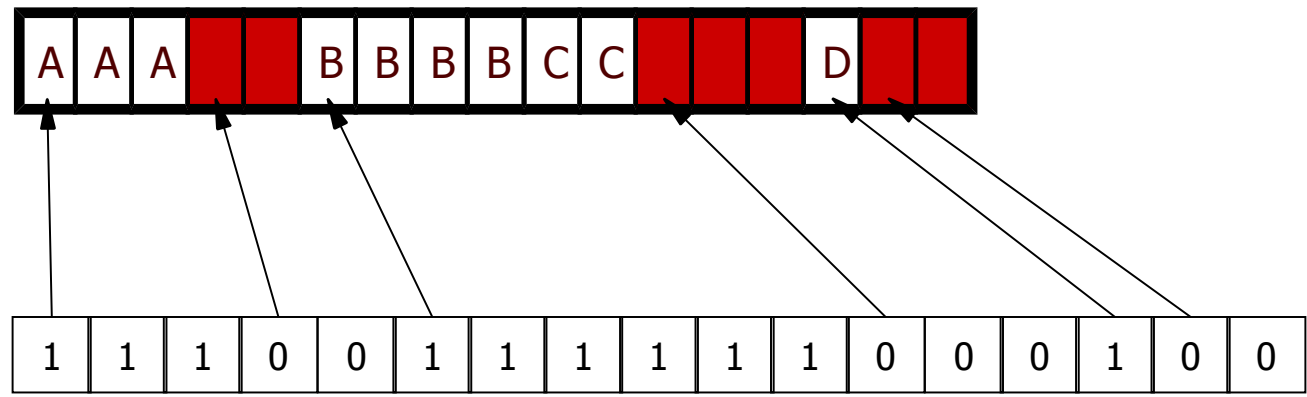
• Listas enlazadas

- Mantener una lista enlazada con los segmentos ocupados y con los segmentos libres. El segmento contendrá un proceso o el hueco existente entre dos procesos.
- La asignación será recorrer la lista buscando un segmento libre igual o mayor que la solicitud.
- La liberación cambiará el valor de ocupado a libre. En el caso de que uno de los dos nodos adyacentes (o los 2) estén libres, se unificarán en uno solo de mayor tamaño.
- Algoritmos de asignación de memoria:
 - El primer hueco donde quepa (*first fit*).
 - El siguiente hueco donde quepa (*next fit*).
 - El hueco donde quepa mejor (*best fit*).
 - El hueco donde quepa antes (*quick fit*).



Tema 5. Sistemas Operativos de Tiempo Real

● Gestión de la memoria dinámica



Tema 5. Sistemas Operativos de Tiempo Real

• Gestión de la memoria dinámica

• Método de los colegas

- La memoria tiene un conjunto de listas de bloques libres de tamaño 1, 2, 4, 8, 16, etc. bytes.
- Las peticiones se ajustan al tamaño inmediatamente superior, particionando (y creando) tantos nodos de tamaño superior o inferior como sean necesarias.
- Cuando se libera suficiente memoria como para reunificar un bloque de tamaño mayor, se eliminan los nodos de tamaños inferiores.
- Sin embargo, aunque haya dos huecos adyacentes, si la unión de ambos no proporciona un hueco de tamaño potencia de dos, no se enlazan.
- Ventaja: rapidez.
- Desventajas: Mala utilización de la memoria: fragmentación interna y externa.



Tema 5. Sistemas Operativos de Tiempo Real

- Gestión de la memoria dinámica
 - Método de los colegas

Inicial	1024															
A: Peticion 70Kb	A	128		256				512								
B: Peticion 35 Kb	A	B	64	256				512								
C: Peticion 80Kb	A	B	64	C	128		512									
Devolución A	128	B	64	C	128		512									
D: Peticion 60Kb	128	B	D	C	128		512									
Devolución B	128	64	D	C	128		512									
Devolución D	256			C	128		512									
Devolución C	1024															

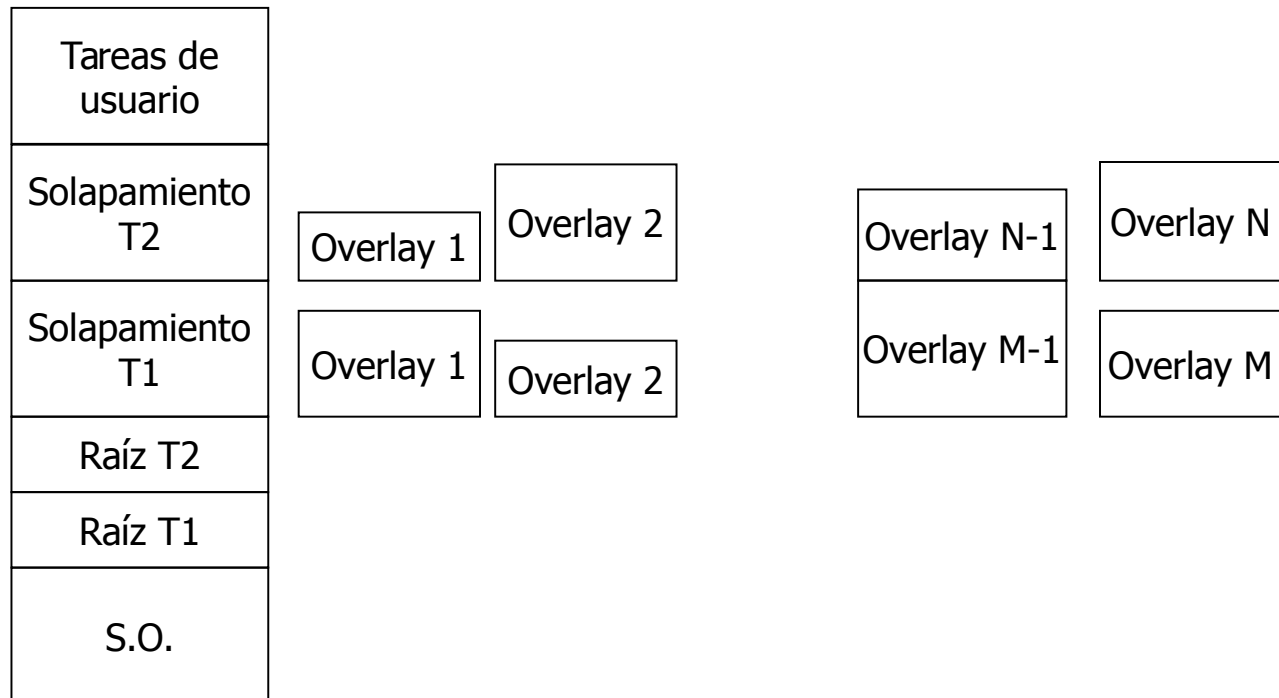
Tema 5. Sistemas Operativos de Tiempo Real

- Intercambio dentro de un programa: encadenamiento y solapamiento
 - Las propias tareas pueden controlar qué bloques de programa se encuentran en cada momento en su zona de memoria asignada.
 - Mecanismo de encadenamiento:
 - La tarea se divide en segmentos que se van ejecutando secuencialmente. Tras la ejecución de un segmento se carga automáticamente el siguiente en el mismo lugar en el que el anterior estaba asignado. Los datos se mantienen en una zona común de la memoria o en disco.
 - Mecanismo de solapamiento:
 - Hay una parte de la tarea (llamada raíz) que siempre se encuentra en memoria y es la encargada de mantener los datos. El resto de segmentos son llevados al área de memoria dedicada al solapamiento.



Tema 5. Sistemas Operativos de Tiempo Real

- Intercambio dentro de un programa: encadenamiento y solapamiento



Tema 5. Sistemas Operativos de Tiempo Real

1. Estructura de los Sistemas de Control

2. Administrador de Tareas

1. Administrador de Memoria

➔ 1. Procesos e Hilos

1. Conceptos básicos sobre procesos

2. Planificación de procesos y de hilos

3. Gestión de Bloqueos en procesos e hilos

4. Reentrancia en procesos e hilos.

1. Comunicación y sincronización entre procesos

2. Control de recursos



Tema 5. Sistemas Operativos de Tiempo Real

- **Procesos e Hilos: Conceptos básicos sobre procesos**
 - Las **tareas** son el conjunto de algoritmos que se ejecutan para cumplir alguna misión u objetivo dentro de un sistema informático.
 - Nos referimos a **programas** a aquellas tareas que no están en ejecución sino almacenados en disco u otro dispositivo de almacenamiento masivo.
 - Las tareas pueden estar compuestas por uno o varios **procesos**. Cada **proceso** está compuesto por:
 - Código ejecutable.
 - Los datos asociados.
 - El contexto de ejecución del proceso (estructuras que sirven para la ejecución del mismo).
 - Los procesos pueden estar compuestos por uno o varios **hilos** (o hebras) de ejecución.



Tema 5. Sistemas Operativos de Tiempo Real

- **Procesos e Hilos: Conceptos básicos sobre procesos**
 - Un proceso tendrá como mínimo un **hilo** de ejecución, sin embargo puede contener varios **hilos** de ejecución.
 - Todos los hilos dentro de un mismo proceso comparten una gran cantidad de variables y estructuras de funcionamiento.
 - Todos los hilos de un proceso comparten el mismo espacio de memoria, tanto datos como código. Cada hilo posee un espacio de pila propio.
 - El cambio de contexto entre los hilos de un mismo proceso es mucho más rápido que entre procesos.



Tema 5. Sistemas Operativos de Tiempo Real

• Planificación de procesos e hilos

- El S.O. gestiona la lista de tareas para ir proporcionando tiempo de CPU a cada una de ellas para ir progresando en su cometido.
- La mayoría de S.O. sólo planifican tiempo de CPU para los procesos, ya que el planificador desconoce la existencia de los hilos y presupone que en el sistema solo existen procesos.
- El cambio de planificación de los procesos es muy costosa, puesto que hay que almacenar muchos registros y variables del proceso que sale de ejecución, y a su vez cargar todos los registros y variables del proceso que entra en ejecución.
- Para evitar tener que hacer los cambios de contexto en el S.O., han implementado hilos que realizan tareas del S.O.



Tema 5. Sistemas Operativos de Tiempo Real

• Planificación de procesos e hilos

- Para evitar tener que hacer los cambios de contexto en el S.O., han implementado hilos que realizan tareas del S.O. que se denominan **hilos kernel**.
- El S.O. tiene conocimiento de la existencia de estos hilos y es capaz de planificar la ejecución de los diferentes hilos.
- Los procesos de usuario también pueden utilizar hilos, denominados **hilos de usuario**. El S.O. no tiene conocimiento de la existencia de estos hilos y por tanto no son planificables directamente.
- El S.O. planifica tiempo de CPU para el proceso completo y éste tiene que repartir el tiempo de CPU entre los diversos hilos del proceso.
- Algunos S.O. proporcionan servicios para crear hilos de usuario que son gestionados y controlados por el propio Sistema Operativo. Estos hilos se denominan **procesos ligeros**.



Tema 5. Sistemas Operativos de Tiempo Real

• Planificación de procesos e hilos

- Los hilos kernel son objetos ligeros planificables y despachables directamente por el S.O.
 - No consumen demasiados recursos del S.O.
 - No necesitan estar asociados a ningún proceso.
 - Comparten espacio de direcciones y recursos del S.O.
 - Pueden ser creados y destruidos a demanda muy rápidamente por el S.O.
 - Pequeño consumo de recursos: pequeña estructura de datos y una pequeña pila.
 - Se suelen utilizar para manejar actividades asíncronas.
 - Se suelen utilizar para dar soporte a los procesos ligeros (cada proceso ligero tiene un hilo kernel asociado, aunque pueden existir hilos kernel no asociados a ningún proceso ligero).

Tema 5. Sistemas Operativos de Tiempo Real

• Planificación de procesos e hilos

- Los procesos ligeros proporcionan múltiples hilos de planificación dentro de un mismo proceso
 - Son planificables independientemente y de manera directa por el S.O.
 - Cada proceso ligero está asociado a un hilo kernel.
 - Elevado consumo de recursos kernel: estructura de datos y memoria para la pila kernel.
 - Comparten algunos recursos comunes entre todos los procesos ligeros de un proceso.
 - Todos los procesos ligeros comparte el conjunto de manejadores de señales, aunque cada uno tiene su propia máscara de señales.
 - No poseen un espacio de nombres global, por lo que son invisibles a otros procesos.
 - Son útiles si tienen poco acceso a datos compartidos ya que la sobrecarga por sincronización anula la ventaja del paralelismo.
 - La creación, destrucción, bloqueo y desbloqueo de los procesos ligeros son llamadas al sistema muy costosas debido al cambio de modo usuario a modo kernel que se produce en cada llamada.
 - Si un proceso tiene muchos procesos ligeros puede llegar a monopolizar la CPU.



Tema 5. Sistemas Operativos de Tiempo Real

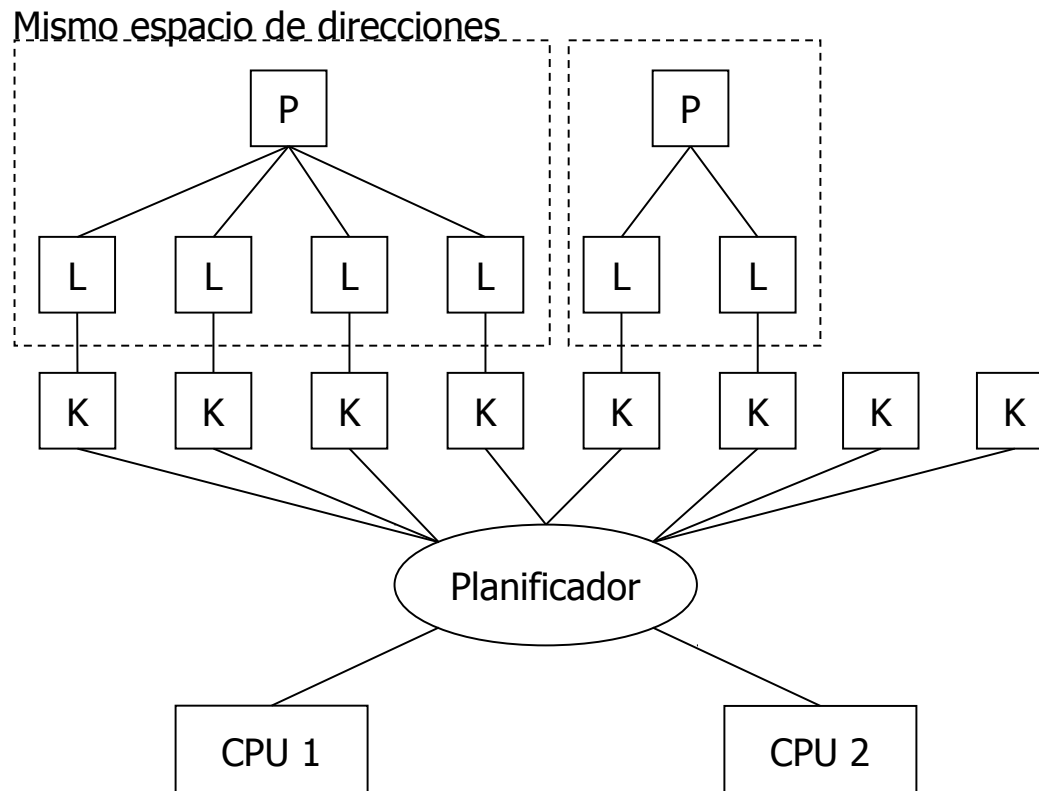
• Planificación de procesos e hilos

- Los hilos de usuario son
 - No son planificables directamente por el S.O. (éste desconoce su existencia).
 - Al no involucrar al S.O. las interacciones entre hilos de usuario son muy rápidas.
 - Cada proceso (usualmente mediante una biblioteca de funciones) tiene un pequeño núcleo/planificador que es el encargado de la planificación de los hilos de usuario.
 - Cada hilo de usuario tiene su propia pila y un pequeño contexto. El consumo de recursos es muy pequeño.
 - Son muy recomendables si hay mucha interacción entre los hilos de usuario y poco acceso a recursos del sistema, puesto que podrían bloquear a todos los restantes hilos.



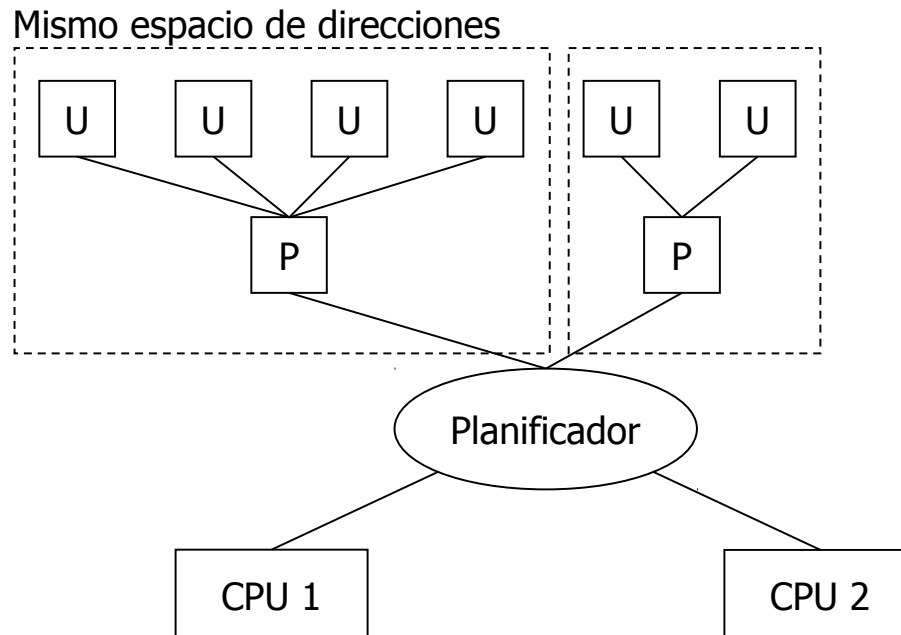
Tema 5. Sistemas Operativos de Tiempo Real

- Planificación de procesos e hilos



Tema 5. Sistemas Operativos de Tiempo Real

- Planificación de procesos e hilos



Tema 5. Sistemas Operativos de Tiempo Real

- Planificación de procesos e hilos
 - Ejemplo de latencia en tiempos en operaciones habituales (tiempos en microsegundos, en una SPARCstation 2 ejecutando Solaris):

	Creación	Sincronización (semáforos)
Hilos Usuario	52	66
Proceso Ligero	350	390
Proceso	1700	200

Tema 5. Sistemas Operativos de Tiempo Real

• Gestión de Bloqueos en procesos e hilos

- El bloqueo de un proceso provoca la retirada de dicho proceso de la lista de "preparados" y se pasa a la lista de "bloqueados".
- En los casos de bloqueo de hilos kernel o de procesos ligeros, el S.O. conoce la existencia de dichos hilos y es capaz de planificar otro hilo diferente que no esté bloqueado, pasando el hilo bloqueado a la lista de "bloqueados".
- En el caso de hilos de usuario, el S.O. no conoce la existencia de dichos hilos y por lo tanto, no puede planificar otro hilo del mismo proceso, sino que bloquea el proceso completo.



Tema 5. Sistemas Operativos de Tiempo Real

• Gestión de Bloqueos en procesos e hilos

- En el caso de hilos de usuario, si un hilo realiza una llamada al sistema bloqueante (por solicitud de un recurso no libre), el S.O. bloqueará el proceso padre, impidiendo la ejecución de ningún otro hilo.
- Si la biblioteca de funciones proporciona mecanismos de comprobación de recurso, sin realizar la solicitud efectiva, podrá impedir el bloqueo de todo el proceso, realizando sucesivas solicitudes al S.O. del recurso hasta que esté libre (mediante llamadas no bloqueantes), planificando internamente otro hilo que no esté bloqueado.
- En el caso de hilos de usuario, los mecanismos de sincronización no podrán ser del S.O. sino de la propia librería, para controlar qué hilos están en ejecución, preparados o bloqueados.



Tema 5. Sistemas Operativos de Tiempo Real

• Reentrancia en procesos e hilos

- Se dice que un código es **reentrante** si admite dos o más llamadas concurrentes al mismo código.
- Todos los códigos que puedan ser llamados por diferentes procesos deben ser reentrantes => Todas las RTI deberían ser reentrantes.
- Para implementaciones del modelo cliente-servidor (o también en otros paradigmas) distintos procesos o hilos pueden realizar llamadas a un mismo código. Este código debe garantizar que las respuestas son correctas en todo caso:
 - Código Reutilizable con Acceso Serializado (Exclusión Mutua)
 - Código Reentrante puro con Acceso concurrente.



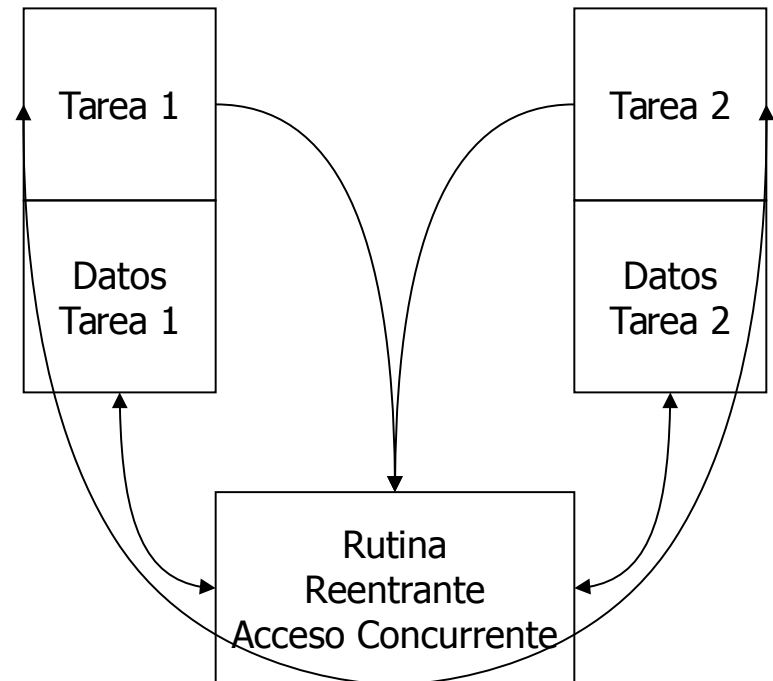
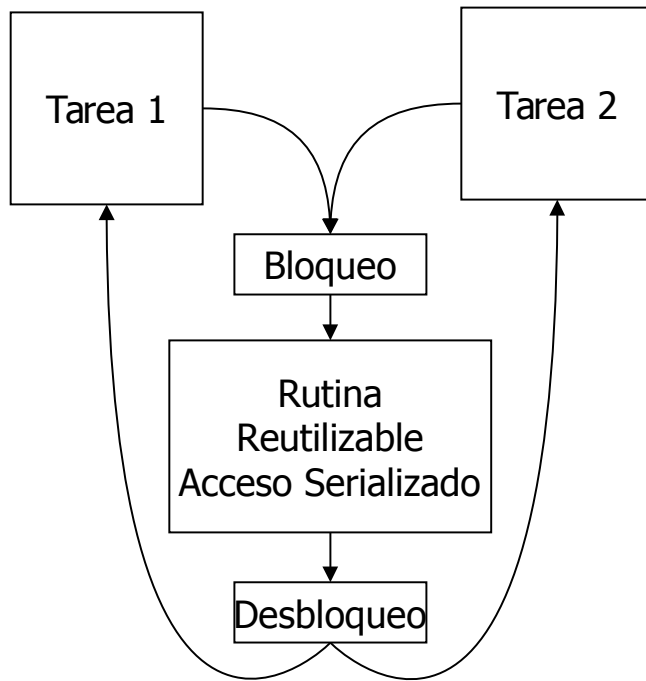
Tema 5. Sistemas Operativos de Tiempo Real

- Reentrancia en procesos e hilos
 - Tareas con **código reutilizable con acceso serializado**.
 - Existe un único código con datos asociados y que su uso está restringido a una tarea cada vez. Estas tareas no pueden ser interrumpidas hasta que finalicen su funcionalidad, tras lo que pueden pasar a servir a otra tarea diferente.
 - Esto se logra con exclusión mutua.
 - Tareas con **código reentrante con acceso concurrente**.
 - Existe un código que no tiene datos asociados o que dentro de su espacio de trabajo no tiene referencias a ninguna tarea en particular. En caso de necesitar almacenar algún dato lo hace en el espacio de memoria de cada tarea llamadora.
 - La reentrancia se consigue mediante el uso de referencias a variables indexadas según la tarea llamadora.



Tema 5. Sistemas Operativos de Tiempo Real

- Reentrancia en procesos e hilos



Tema 5. Sistemas Operativos de Tiempo Real

1. Estructura de los Sistemas de Control
2. Administrador de Tareas
1. Administrador de Memoria
1. Procesos e Hilos
- 2. Comunicación y sincronización entre procesos
 1. Señales: generación, manejo, sincronización
 2. Exclusión mutua con espera ocupada
 3. Sincronización de "larga duración": Dormir/Despertar (sleep/wakeup)
 4. Soporte de semáforos y monitores en Sistemas en Tiempo Real
1. Control de recursos



Tema 5. Sistemas Operativos de Tiempo Real

• Señales: generación, manejo y sincronización

- El S.O. es el encargado de proporcionar mecanismos a las tareas para generar señales que les permitan comunicarse con otras y con el propio S.O.
- El S.O. también utilizará las señales para comunicarse con las tareas e informarles de algunos eventos o de excepciones.
- El manejo de las señales se realiza a nivel de entidad planificable (procesos, procesos ligeros o hilos kernel).
- El S.O. es el encargado de enviar las señales a los procesos, indicar que tienen una señal pendiente y de proporcionar los datos para que los procesos puedan obtenerlas.
- La sincronización se realiza a nivel de S.O., siendo responsable éste de seleccionar la tarea receptora y desbloquearla (pasarla a la lista de preparados).



Tema 5. Sistemas Operativos de Tiempo Real

• Exclusión mutua con espera ocupada

- Para sincronizaciones de corta duración, la mejor opción es la espera ocupada.
- En estos casos, el S.O. no saca a la tarea (proceso/hilo) de la cola de "preparados", ya que consumen tiempo de CPU chequeando la condición.
- Al no estar bloqueado, el S.O. no satura la cola de "bloqueados" y no tiene que buscar la tarea para despertarla.
- Este mecanismo no garantiza que, una vez la condición sea cierta, la tarea en espera ocupada pueda continuar inmediatamente, sino que tendrá que esperar a que sea planificado de nuevo y entre en ejecución.
- No son útiles si la activación de la condición tarda en realizarse mucho tiempo (varios cientos de ciclos o más).



Tema 5. Sistemas Operativos de Tiempo Real

- **Sincronización de "larga duración": Dormir-Despertar**
 - El S.O. es el encargado de la gestión de dormir/despertar a las tareas.
 - Cuando una tarea solicita "dormir", el S.O. la pasa a la cola de "bloqueados".
 - Cuando una tarea desea "despertar" a otra, el S.O. busca dicha tarea en la cola de "bloqueados" y la pasa a la cola de "preparados".
 - El S.O. hace uso del sistema de señales entre tareas para despertar la tarea deseada.
 - Los mecanismos dormir/despertar consumen mucho tiempo de CPU debido a la gran cantidad de llamadas al sistema que involucran.
 - No son útiles si se espera que la activación se realice en poco tiempo (unos pocos cientos de ciclos).



Tema 5. Sistemas Operativos de Tiempo Real

• Soporte de semáforos y monitores en SOTR

- En los SOTR, el uso de semáforos y monitores está muy penalizado debido al alto consumo computacional que suelen involucrar.
- Los semáforos (y los monitores) presentan algunos problemas:
 - Para garantizar la atomicidad, se aumenta temporalmente el nivel de prioridad al máximo de la tarea llamadora => Así no será desplanificada:
 - Problemas con tareas de alta prioridad REAL.
 - Necesidad de un S.O. que pueda cambiar dinámicamente las prioridades de las tareas.
 - Para evitar la actuación de las interrupciones se inhiben interrupciones temporalmente => Problemas con interrupciones y eventos de alta prioridad.
 - Los semáforos no están adscritos a ningún proceso => el S.O. tiene que crear hilos kernel o procesos y consumir espacio del S.O. para su manejo.
- Los semáforos impiden un control exhaustivo del tiempo de cómputo que puede llevar al incumplimiento de *deadlines* de tareas de alta prioridad.



Tema 5. Sistemas Operativos de Tiempo Real

1. Estructura de los Sistemas de Control
2. Administrador de Tareas
1. Administrador de Memoria
1. Procesos e Hilos
2. Comunicación y sincronización entre procesos
3. Control de recursos
 1. Utilización de recursos compartidos: acceso compartido, exclusivo, mixto
 2. Conceptos: "Deadlocks", "Livelocks", bloqueo indefinido
 3. Condiciones necesarias para un "Deadlock"
 4. Prevención del "Deadlock": Algoritmo del banquero, detección y recuperación
 5. Subsistema de Entrada/Salida



Tema 5. Sistemas Operativos de Tiempo Real

Control de recursos

Según el tipo de acceso:

Acceso compartido

- El recurso puede ser compartido por varias tareas simultáneamente.

Acceso exclusivo

- Una única tarea puede acceder a un recurso a la vez.

Acceso mixto

- El recurso puede ser accedido algunas veces de manera compartida y otras veces de manera exclusiva.

Utilización de recursos compartidos:

- Solicitud del recurso.
- Utilización del recurso.
- Liberación del recurso.



Tema 5. Sistemas Operativos de Tiempo Real

Control de recursos

- Si un recurso está siendo utilizado en modo exclusivo y un segundo proceso intenta acceder a dicho recurso en modo compartido, éste debe esperar.
- Si un recurso está siendo utilizado en modo compartido y otro proceso intenta acceder a dicho recurso en modo exclusivo, éste deberá esperar a que todos los procesos que están accediendo en modo compartido liberen su utilización.
 - El S.O. debe tener conocimiento de que hay, al menos, un proceso solicitando acceso en modo exclusivo, para impedir que nuevos procesos accedan al recurso en modo compartido.
- Como recomendación general, debido a que los procesos pueden bloquearse cuando solicitan un recurso, no deben pedirlo hasta justo cuando lo necesiten y liberarlo tan pronto como sea posible.



Tema 5. Sistemas Operativos de Tiempo Real

- “Deadlocks”, “Livelocks”, bloqueo indefinido
 - El sistema puede estar falto de viveza por alguna de estas causas:
 - **Deadlock**
 - Se produce cuando dos procesos no pueden progresar porque necesitan un recurso que posee el otro, la solicitud de los recursos produce el bloqueo de cada proceso, haciendo que no se ejecuten nunca. Ambos procesos estarán esperando bloqueados a que el otro proceso libere el recurso que necesita.
 - **Livelock**
 - Similar a un *deadlock*, pero las tareas no se bloquean, solo que se mantienen iterando (consumiendo tiempo de CPU) sin que ninguna consiga progresar.
 - **Bloqueo indefinido**
 - Cuando varios procesos intentan acceder a un recurso en exclusiva y nunca lo consiguen. Esto puede ser debido a la política de asignación de recursos (si no impide las solicitudes de recursos en modo compartido una vez hay una solicitud de modo exclusivo).



Tema 5. Sistemas Operativos de Tiempo Real

- Condiciones necesarias para un "Deadlock"
 - Para que se produzca un *deadlock* se tienen que dar 4 condiciones:
 - Exclusión Mutua.
 - Solo un proceso puede utilizar un recurso a la vez.
 - Mantenimiento y espera.
 - Un proceso adquiere un recurso mientras que el resto están a la espera de obtenerlo.
 - No apropiación.
 - Un recurso no puede ser liberado por el S.O. o por otro proceso salvo por el propio proceso que lo ha adquirido.
 - Espera circular.
 - Existe una encadenamiento de procesos esperando, de tal forma que cada proceso mantenga recursos que están siendo solicitados por el siguiente proceso en la cadena.
 - Si no se dan las 4 condiciones, no se producirá un *deadlock*.



Tema 5. Sistemas Operativos de Tiempo Real

- **Mecanismos para no incurrir en un "Deadlock"**
 - Ante el problema de los *deadlock* hay 3 mecanismos:
 - Prevenir los *deadlocks*.
 - Evitar entrar en una situación que pueda llevar a un *deadlock*.
 - Detectar si se ha entrado en un *deadlock* y recuperarse de él antes de que se produzca el bloqueo efectivo.
 - Los dos primeros mecanismos son fácilmente implementables: el primero implica al programador, el segundo implica al S.O.
 - El tercer mecanismo es una técnica basada en la teoría de grafos, que trata de comprobar si un grafo es reducible en cada momento y si no lo es, proponer un mecanismo para resolverlo.
 - Es muy complicado y no se implementa en la realidad, solo se utiliza a nivel teórico.



Tema 5. Sistemas Operativos de Tiempo Real

• Prevención de "Deadlock"

- Para evitar que se produzca un *deadlock* hay que conseguir que no den alguna de las 4 condiciones:
 - Exclusión mutua:
 - Para evitar esta condición se puede imponer que el acceso sea compartido o restringir la utilización del recurso en exclusiva a un único proceso.
 - Mantenimiento y espera:
 - Hacer que los procesos soliciten los recursos antes de comenzar a ejecutarse o en los instantes de ejecución que no tengan recursos pedidos. Esto provoca un uso muy ineficiente de los recursos e incluso al bloqueo en ciertos casos.
 - No apropiación:
 - Permitir que el S.O. pueda apropiarse de los recursos de un proceso que no responde en un determinado tiempo. Es muy complicado de implementar: requiere guardar el estado antes y después de la apropiación, hecho que a veces es imposible.
 - Espera circular:
 - Imponer un orden lineal de acceso a los recursos y no admitir solicitudes de un proceso que tenga un recurso adquirido y solicite un recurso que tenga un orden lineal inferior al del recurso ya adquirido.



Tema 5. Sistemas Operativos de Tiempo Real

• Prevención de "Deadlock"

- Evitar la Espera circular mediante acceso lineal.
 - Definir el conjunto de recursos del sistema como:

$$R = \{r_1, r_2, r_3 \dots r_n\}$$
 - Definir una función de acceso lineal que devuelve el orden de un recurso en la lista de recursos del sistema:

$$F(r_i) = i$$
 - Si un proceso tiene un recurso r_j y solicita el recurso r_k se proporcionará acceso a dicho recurso solo si $F(r_j) < F(r_k)$.
 - O lo que es equivalente, un proceso que solicite el recurso r_j deberá liberar antes todos los recursos $r_i, i = 1 \dots k-1$; tal que $F(r_i) < F(r_k)$.



Tema 5. Sistemas Operativos de Tiempo Real

- Evitar "Deadlock" usando el algoritmo del banquero
 - Diseñado por Dijkstra (1965), se basa en el sistema bancario para la previsión del uso del límite del crédito máximo y sirve para evitar entrar en una situación que pudiera finalizar en un *deadlock*.
 - Se supone que se tienen varias unidades de un mismo recurso.
 - Un estado es seguro si puede atender al menos 1 solicitud de recursos máximo por parte de un proceso.
 - Un estado es inseguro si no puede atender ninguna solicitud del máximo de recursos por parte de ninguno de los procesos.

Disp. 10	Utilizados	Necesita
P1	0	6
P2	0	5
P3	0	4
P4	0	7

Disp. 2	Utilizados	Necesita
P1	1	6
P2	1	5
P3	2	4
P4	4	7

Estado seguro

Disp. 1	Utilizados	Necesita
P1	1	6
P2	2	5
P3	2	4
P4	4	7

Estado inseguro

Tema 5. Sistemas Operativos de Tiempo Real

● Subsistema de Entrada/Salida

- El S.O. debe manejar las peculiaridades de cada dispositivo y presentar a los procesos un mecanismo de acceso unificado a todos los dispositivos.
- El S.O. también debe sincronizar los accesos concurrentes a los dispositivos.
- Los programadores no deberían acceder directamente a los dispositivos, sino a través de llamadas al S.O. para evitar que éste no controle los accesos.
- Un acceso directo a un dispositivo puede ser más rápido al "saltarnos" el nivel de S.O., sin embargo podría dejar inconsistente el S.O. o podrían producirse efectos no controlados.



Tema 5. Sistemas Operativos de Tiempo Real

● Subsistema de Entrada/Salida

- El Subsistema de E/S (en inglés, IOSS: *Input Output SubSystem*) está dividido en 2 niveles y 3 módulos:
 - Nivel de interfaz con el S.O.
 - El manejador de E/S.
 - Nivel de enlace con el dispositivo.
 - El Bloque de control de dispositivo (en inglés, DCB: *Device Control Block*).
 - Los *drivers* del dispositivo.
- Hay un único manejador de E/S que acepta las llamadas al sistema que realizan las tareas de usuario para solicitar acceso a dispositivos de E/S.
- El manejador de E/S transfiere la información de la llamada de la tarea de usuario al DCB que es el encargado de procesar dicha llamada y traducirla a funciones específicas del dispositivo.
- Los *drivers* del dispositivo contienen las implementaciones de las funciones específicas que permiten acceder al hardware.



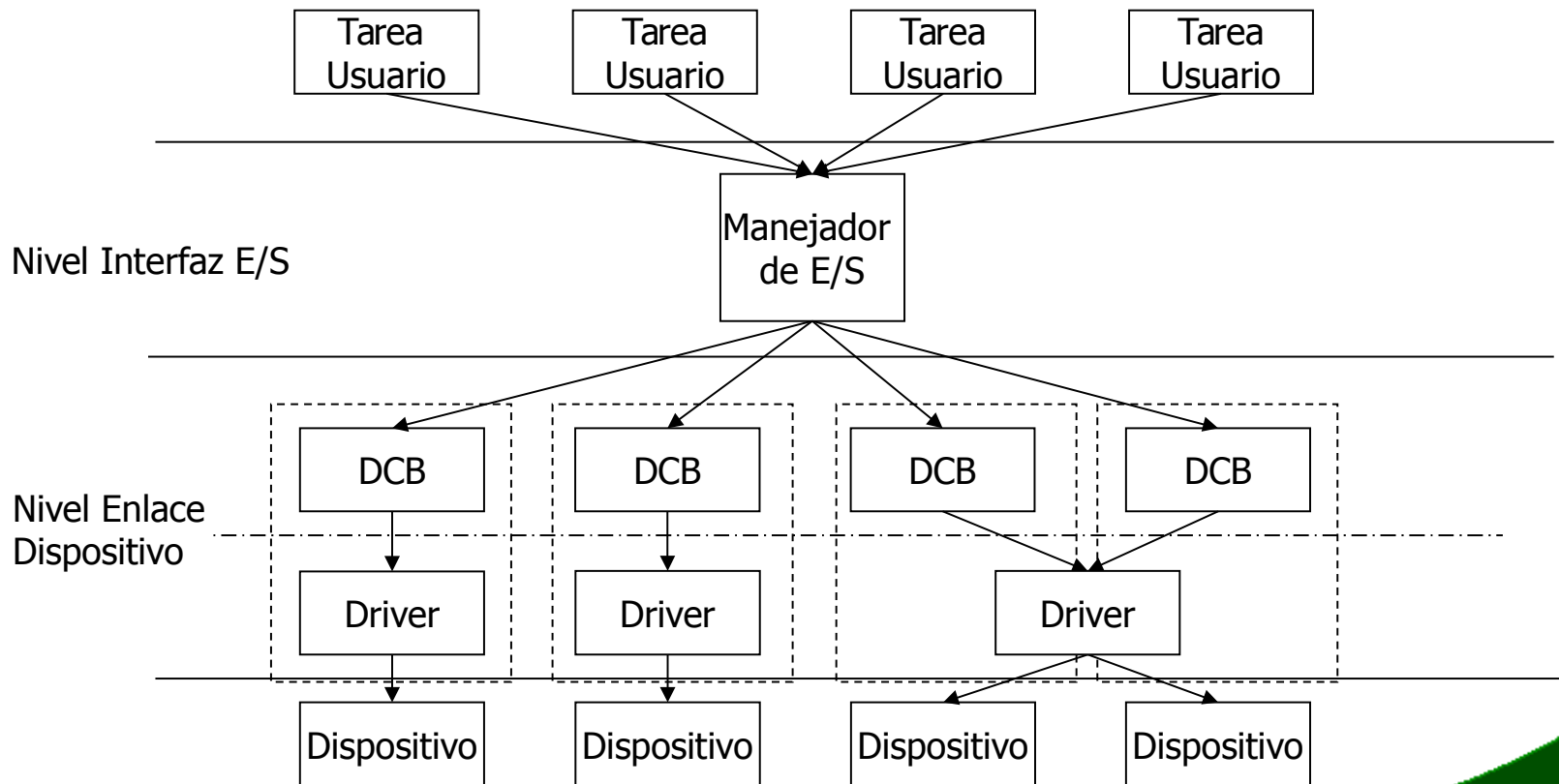
Tema 5. Sistemas Operativos de Tiempo Real

● Subsistema de Entrada/Salida

- Existirá un DCB por cada instancia de dispositivo existente en el sistema.
- Podrá existir *drivers* específicos por cada dispositivo o incluso se podrá compartir un mismo *driver* entre varios dispositivos similares, aunque cada uno tendrá un DCB diferente.
- La transferencia de datos lo realiza el *driver*, mediante interrupciones, utilizando la información en el DCB asociado.
- Los DCB suelen almacenar información de:
 - Relativa al dispositivo
 - Nombre del dispositivo físico y Tipo de dispositivo
 - Dirección del dispositivo
 - Interrupción asociada y dirección de la RTI asociada
 - Estado del Dispositivo
 - Relativo a los datos
 - Dirección del búfer de datos
 - Tipo de transferencia (binaria/ASCII)
 - Cantidad de bytes a transferir y Contador actual de transferencia

Tema 5. Sistemas Operativos de Tiempo Real

● Subsistema de Entrada/Salida



Preguntas ...

Volver

